

```
#!/usr/bin/env python3
"""
Jovem (Y1,Y2,Y3) vs. Idoso (O1,O2) – HRV
real (Fantasia/PhysioNet) + 30 sementes
"""

import numpy as np
from scipy.interpolate import interp1d
from scipy import stats

K = 4
T = 400
kappa = 0.25
precision_ema = 0.05
ignition_threshold = 2.5
broadcast_gain_normal = 0.5
obs_noise = 0.3
precision_ceiling_normal = 5.0
N_SEEDS = 30

def true_signal(k, t):
    base_freqs = [0.02, 0.035, 0.015, 0.05]
    event_times = [100, 180, 260, 320]
    val = np.sin(2 * np.pi * base_freqs[k]
    * t)
    if t > event_times[k]:
        val += 1.5
    return val

def sliding_rmssd(rr_series, window=60,
step=30):
```

```

        diffs = np.diff(rr_series) * 1000
        vals = []
        for i in range(0, len(diffs) - window,
step):
            w = diffs[i:i+window]
            vals.append(np.sqrt(np.mean(w**2)))
        return np.array(vals)

def hrv_to_refractory(hrv_series,
min_ref=8, max_ref=30):
    hrv_norm = (hrv_series -
hrv_series.min()) / (hrv_series.max() -
hrv_series.min() + 1e-9)
    return np.round(max_ref - (max_ref -
min_ref) * hrv_norm).astype(int)

def load_subject(path):
    rr = np.loadtxt(path)
    rmssd = sliding_rmssd(rr)
    x_old = np.linspace(0, 1, len(rmssd))
    x_new = np.linspace(0, 1, T)
    rmssd_interp = interp1d(x_old, rmssd,
kind='linear')(x_new)
    return rr, rmssd, rmssd_interp

base =
'/home/claude/active_inference_workspace/'
subjects_young =
['fantasia_Y1_RR_intervals.txt',
'fantasia_Y2_RR_intervals.txt',
'fantasia_Y3_RR_intervals.txt']
subjects_old =
['fantasia_O1_RR_intervals.txt',
'fantasia_O2_RR_intervals.txt']

```

```

print("="*70)
print("HRV REAL: JOVEM (Y1,Y2,Y3) vs. IDOSO
(01,02) – Fantasia/PhysioNet")
print("="*70)

young_data = []
for f in subjects_young:
    rr, rmssd, hrv_interp =
load_subject(base + f)
    print(f"{f}: RMSSD medio =
{rmssd.mean():.1f} ms")
    young_data.append(hrv_interp)

old_data = []
for f in subjects_old:
    rr, rmssd, hrv_interp =
load_subject(base + f)
    print(f"{f}: RMSSD medio =
{rmssd.mean():.1f} ms")
    old_data.append(hrv_interp)

print(f"\nMedia jovem (n=3):
{np.mean([d.mean() for d in
young_data]):.1f} ms")
print(f"Media idoso (n=2):
{np.mean([d.mean() for d in old_data]):.1f}
ms")

def run_simulation(seed,
refractory_series=None,
refractory_fixed=15):
    np.random.seed(seed)
    mu = np.zeros(K)

```

```

precision = np.ones(K) * 1.0
err_ema_sq = np.ones(K) * 1.0
refractory_until = np.zeros(K,
dtype=int)
ignitions = 0
abs_err_sum = 0.0

for t in range(T):
    ref_period = refractory_series[t]
    if refractory_series is not None else
    refractory_fixed
        true_vals =
np.array([true_signal(k, t) for k in
range(K)])
        obs = true_vals +
np.random.randn(K) * obs_noise
        err = obs - mu
        salience = precision * err**2
        mu = mu + kappa * precision * err
        err_ema_sq = (1 - precision_ema) *
err_ema_sq + precision_ema * err**2
        precision = np.clip(1.0 / (0.1 +
err_ema_sq), 0.1, precision_ceiling_normal)
        eligible = np.array([salience[k] if
t >= refractory_until[k] else -1.0 for k in
range(K)])
        winner = int(np.argmax(eligible))
        ignite = eligible[winner] >
ignition_threshold
        if ignite:
            ignitions += 1
            refractory_until[winner] = t +
ref_period
            for j in range(K):

```

```

        if j != winner:
            mu[j] = mu[j] +
broadcast_gain_normal * (mu[winner] -
mu[j])
            abs_err_sum += np.mean(np.abs(mu -
true_vals))

    return ignitions, abs_err_sum / T

def run_multi_seed(refractory_series=None,
refractory_fixed=15, n_seeds=N_SEEDS):
    ign_list, err_list = [], []
    for s in range(n_seeds):
        ign, err =
run_simulation(seed=1000+s,
refractory_series=refractory_series,
refractory_fixed=refractory_fixed)
        ign_list.append(ign)
        err_list.append(err)
    return np.array(ign_list),
np.array(err_list)

print("\n" + "="*70)
print(f"RODANDO {N_SEEDS} SEMENTES POR
SUJEITO")
print("="*70)

ign_fixed, err_fixed =
run_multi_seed(refractory_series=None)

young_errs, young_igns = [], []
for i, hrv in enumerate(young_data):
    refr = hrv_to_refractory(hrv)
    ign, err =

```

```

run_multi_seed(refractory_series=refr)
    young_errs.append(err)
    young_igns.append(ign)
    print(f"Jovem {i+1}: ignicoes media=
{ign.mean():.1f} erro medio=
{err.mean():.4f}")

old_errs, old_igns = [], []
for i, hrv in enumerate(old_data):
    refr = hrv_to_refractory(hrv)
    ign, err =
run_multi_seed(refractory_series=refr)
    old_errs.append(err)
    old_igns.append(ign)
    print(f"Idoso {i+1}: ignicoes media=
{ign.mean():.1f} erro medio=
{err.mean():.4f}")

# media por semente entre sujeitos do mesmo
grupo
err_young_avg = np.mean(young_errs, axis=0)
ign_young_avg = np.mean(young_igns, axis=0)
err_old_avg = np.mean(old_errs, axis=0)
ign_old_avg = np.mean(old_igns, axis=0)

print("\n" + "="*70)
print("RESUMO POR GRUPO (media entre
sujeitos, 30 sementes)")
print("="*70)
print(f"REFRATARIO FIXO:          ignicoes=
{ign_fixed.mean():.1f}+-
{ign_fixed.std():.1f} erro=
{err_fixed.mean():.4f}+-
{err_fixed.std():.4f}")

```

```

print(f"GRUPO JOVEM (n=3):          ignicoes=
{ign_young_avg.mean():.1f}+-
{ign_young_avg.std():.1f} erro=
{err_young_avg.mean():.4f}+-
{err_young_avg.std():.4f}")
print(f"GRUPO IDOSO (n=2):          ignicoes=
{ign_old_avg.mean():.1f}+-
{ign_old_avg.std():.1f} erro=
{err_old_avg.mean():.4f}+-
{err_old_avg.std():.4f}")

print("\n" + "="*70)
print("TESTES ESTATISTICOS (pareado por
semente)")
print("="*70)
t1, p1 = stats.ttest_rel(err_young_avg,
err_old_avg)
print(f"Erro: jovem vs idoso -> t={t1:.3f},
p={p1:.4f}")
t2, p2 = stats.ttest_rel(ign_young_avg,
ign_old_avg)
print(f"Ignicoes: jovem vs idoso -> t=
{t2:.3f}, p={p2:.4f}")
t3, p3 = stats.ttest_rel(err_fixed,
err_young_avg)
print(f"Erro: fixo vs jovem -> t={t3:.3f},
p={p3:.4f}")
t4, p4 = stats.ttest_rel(err_fixed,
err_old_avg)
print(f"Erro: fixo vs idoso -> t={t4:.3f},
p={p4:.4f}")

```

